LCD 屏幕测试程序（第二版 `lcd_test2`）说明

> 目标：一块板子插上屏之后，用**一套程序**完成"屏好不好 + 触摸好不好"的装配后快速验证： **标准色轮播（F1）+ 触摸点位/实时反馈（F2/F3）+ 按键切模式（F4）+ 串口遥测（F5）**。 上一版触摸实验程序完整保留在 `lcd_test/`，本版是**独立目录、独立顶层、独立 XDC、独立位流**，两者互不引用。

---

1. 文件清单

文件	说明
`lcd_test2/rtl/lcd_test2_top.v`	顶层：引脚、时序发生器、键盘、触摸、画面、遥测装配
`lcd_test2/rtl/screen_pattern.v`	**新增**：F1 标准色 + F2 网格/刻度/信息面板 + F3 准星
`lcd_test2/rtl/key_ctrl.v`	**新增**：按键消抖 + 短按/长按判定（F4）
`lcd_test2/rtl/touch_i2c_ctrl.v`	由 `lcd_test` 复制后按 F6 改造（按需读 + 降速轮询 + INT 快通道）
`lcd_test2/rtl/lcd_timing_gen.v`	由 `lcd_test` 原样复制（1024×600@59 时序）
`lcd_test2/rtl/i2c_master_seq.v`	由 `lcd_test` 复制后**修过看门狗**（按状态停留计时 + 新增 `S_RECOVER` 总线恢复 + `dbg_state/dbg_busy` 观测口，见 §6 第 6 条与 §9.7）
`lcd_test2/rtl/lcd_test_support.v`	由 `lcd_test` 复制，**追加** `uart_frame_sender26`（v3 26 字节帧）
`lcd_test2/constr/lcd_test2.xdc`	与 `lcd_test/constr/lcd_test.xdc` **逐字节相同**（端口未变，已用哈希核对）
`lcd_test2/sim/*.v`	8 个自检测试台 + 2 个仿真模型（`iic_slave_model.v`、`iobuf_stub.v`）
`lcd_test2/tools/{sim,synth,impl,program}_lcd_test2.tcl`	仿真 / 综合 / 实现 / 下载脚本
`lcd_test2/tools/lcd2_uart_check.py`	上位机：读 COM 口、逐字节解 v3 帧并打印
`lcd_test2/tools/lcd2_board_check.py`	**上位机：上板引导式验收**——一步步提示你按键/点屏，自动采集遥测并判定，结果写 `_lcd_work/board_check_result.md`
`lcd_test2/tools/lcd2_touch_probe.py`	**上位机：触摸专项四阶段探测**（静止基线 → 单指 → 抬手 → 多指），逐帧打印 `0x814E` 原始字节与点数直方图，用来分清"芯片不报点"还是"RTL 解码错"（见 9.6）
`docs/LCD_test_program_requirements.md`	需求书（V1 稿，用户已确认"按建议来"）
本文件	说明 + 判据 + 验证记录

---

2. 引脚（与 `lcd_test` 完全一致，来自开发指南表 2.1.1）

信号	管脚	方向	说明
`sys_clk`	V4	in	50 MHz 晶振
`key_n`	W9	in	板载按键，低有效
`uart_tx`	W21	out	115200 8N1 遥测
`lcd_rgb[23:0]`	—	inout	高 8=R、中 8=G、低 8=B；其中 `[7]/[15]/[23]` 需双向读屏 ID
`lcd_pclk / lcd_hs / lcd_vs / lcd_de`	U1 / AB2 / T1 / W2	out	像素时钟与同步
`lcd_bl / lcd_rst`	Y2 / R2	out	背光（常亮）/ 面板复位
`touch_scl / touch_sda`	T4 / U5	inout	开漏 IIC（板上有上拉，XDC 里不加 PULLUP）
`touch_int`	T5	**inout**	触摸中断（低有效）。复位期间 FPGA 把它拉低，GT911 在 RST 上升沿按 INT 电平锁存器件地址（定死在 `0x5D`），之后释放成高阻靠内部 PULLUP 拉高，芯片才能用低脉冲报"有数据"
`touch_rst_n`	W4	out	触摸复位（低有效）

⚠ 上电前先插好 40P 排线；`lcd_rgb[7]/[15]/[23]` 在读 ID 期间为高阻（XDC 已按 IOBUF 处理）， 否则会和屏模块的电阻打架、读不到 ID。综合脚本会打印 `IOBUF_CHECK` 数量供核对。

---

3. 操作与模式（F4）

操作	效果
上电	默认进 **F1 标准色轮播**（一眼看屏）
短按 `key_n`（< 1 s）	`F1 色轮播 ↔ F2/F3 网格 + 准星`
长按 `key_n`（≥ 1 s）	`paused` 取反：冻结色轮播 / 冻结准星保留计数

按键参数：消抖 10 ms（`DB_CYCLES=500_000`），长按阈值 1 s（`LONG_CYCLES=50_000_000`）。 `key_n` **不再**复位 LCD（`lcd_rst_n` 只由读 ID 模块驱动），避免按键把屏闪一下。

---

4. 画面

F1 标准色轮播（`mode=0`）

- 8 色循环：**白 → 黑 → 红 → 绿 → 蓝 → 青 → 品红 → 黄**，整屏满幅。
- 每色 **58 帧 ≈ 0.99 s**（帧率 50 MHz/(1344×635) ≈ 58.6 Hz，`COL_FRAMES` 可配）。
- 四角小字（都带黑底，保证压在纯色上可读）：

- 左上：色名 + 序号，如 `WHITE 1/8`
- 右上：RGB888 十六进制，如 `FFFFFF`
- 右下：`FRAME nnnnnn`（连续帧计数低 6 位）
- 用途：肉眼查坏点、色偏、背光均匀性、24 位数据线接反/错位。

F2 触摸点位（`mode=1`）

- 背景：**64 px** 坐标网格（`#2A3A4A`），每 **128 px** 画一条主网格（`#4A6A8A`），非网格底色 `#101820`。
- 刻度：底部每 128 px 标 X，右侧每 128 px 标 Y（3 位十进制，白字）。
- 信息面板：左上 (8,8) 起 320×128，20 列 × 8 行、5×7 字模放大 2 倍，白字黑底，**最后绘制**保证可读：

`TOUCH TEST + DOWN PTS n IC GT911 STS xx INT n Vn T1 xxxx,yyyy ...`（最多 5 行）

F3 实时准星（`mode=1` 且有触点）

- 触点处画**十字（±16 px 臂宽 1）+ 空心方框（±5，内部空 ±3）**。
- 5 点颜色固定：**T1 白 / T2 红 / T3 绿 / T4 蓝 / T5 黄**；在触点右下方 `(+14,+14)` 画 2 字符编号 `T1..T5`。
- 抬手后准星**保留 30 帧 ≈ 0.5 s**（`HOLD_FRAMES` 可配），便于确认最后落点。
- 按下到画出目标 ≤ 1 帧：画面与触摸读取都在 `frame_st` 处更新，触点数据当帧即可用。

---

5. 串口遥测协议 v3（F5）

`115200 8N1`，**每秒 1 帧，26 字节，大端**：

偏移	字段	说明
0–2	`A5 4C 03`	帧头 + 版本
3	`id_bits`	`{id_valid, 2'b00, one-hot[4:0]}`（读屏 ID 结果）
4	`{tstate[2:0], step[3:0], mode}`	低 1 位 = `mode`（0 = F1，1 = F2/F3）；高 7 位是触摸状态机诊断字段（见下）
5	`{6'd0, m_busy, paused}`	低 1 位 = `paused`；bit1 = IIC 主机 `busy`
6	`{2'd0, m_state[3:0], tch_type[1:0]}`	低 2 位 = `tch_type`（0 = 未识别，1 = GT911，2 = FT5206）；高 4 位 = IIC 主机 `state`
7	`tch_valid`	当前有有效触点
8	`tch_num`	点数 0..5
9	`gt_status_raw`	GT911 `0x814E` 原始值（诊断点 0 问题的关键字节）
10	`color_idx`	当前色号 0..7
11–13	`frame_cnt[23:0]`	连续帧计数
14–15 / 16–17	`T1 x / T1 y`	第 1 点坐标
18–19 / 20–21	`T2 x / T2 y`	第 2 点坐标

偏移	字段	说明
22–23	`reads_ps`	本秒 IIC 传输次数（看总线占用）
24–25	`err_ps`	本秒 NACK + 超时次数（看总线健康）

上位机：`E:\Anaconda\python.exe lcd_test2\tools\lcd2_uart_check.py COM3`

5.1 触摸状态机诊断字段（2026-10-15 新增）

上一版出现"一碰屏就永久停摆"（`reads_ps` 掉 0、所有触摸字段冻结、`err_ps` 也是 0）， 但从遥测只能看到结果、看不到原因。于是把 4/5/6 字节里原本的空位利用起来上报两套状态机：

字段	位	含义
`mode`	byte4[0]	画面模式
`step`	byte4[4:1]	`touch_i2c_ctrl` 的步骤：0 `GT_PB`、1 `FT_PB`、2 `FT_PB2`、3 `LOOP`、4 `FT_RD`、5 `CLR`、6 `GT_ID`、7 `GT_INFO`、8 `POLL`、9 `CFG`、10 `LOOP4`
`tstate`	byte4[7:5]	`touch_i2c_ctrl` 的状态：0 `IDLE`、1 `REQ`、2 `WAIT`、3 `END`
`paused`	byte5[0]	长按冻结
`m_busy`	byte5[1]	IIC 主机 `busy`
`tch_type`	byte6[1:0]	触摸型号
`m_state`	byte6[5:2]	IIC 主机状态：0 `IDLE`、1 `START`、2 `SEND_ADDR`、3 `ACK_ADDR`、4 `SEND_TX`、5 `ACK_TX`、6 `RX`、7 `ACK_RX`、8 `STOP`、9 `END`、10 `RECOVER`

> ★ 2026-10-15 修正：`dbg` 总线里 `step` 在 `dbg[11:8]`、`m_state` 在 `dbg[7:4]`，而顶层最初按 `dbg[10:7]` / `dbg[6:4]` 打包，导致串口里的 `step` 恒显示 `GT_PB(0)`、`m_state` 恰好只对 0..7 有效。已改正（`lcd_test2_top.v` 的 byte4/byte6 打包 + 上位机 `m_state` 掩码由 0x7 改 0xF）。

怎么看：正常空闲时 `tstate=IDLE / step=POLL`，偶发 `step=LOOP(3)` 或 `LOOP4(10)`； 一轮多点读完成会短暂经过 `REQ/WAIT`。若一屏帧里齐刷刷是 `tstate=WAIT + m_busy=1 + m_state=某相位`，就说明**上层在等 `done`、而 IIC 主机卡在相位里** ——这正是上一版的故障形态。`m_state=RECOVER(10)` 说明看门狗触发过、正在打时钟救总线。

`lcd2_uart_check.py` 会逐帧打印这一行，并在 window 汇总里给出状态分布直方图。

---

6. 触摸底层改造（F6，本版重点）

上一版结论：GT911 能识别、产品 ID `0x39`、配置分辨率 1024×600、IIC 与 INT 电平正常， 但 `0x814E` 恒为 `0x00`（点数 0）。本版做了三件事：

1. **按需读**：新增 `STEP_POLL` 状态，每轮只读状态字节 `0x814E`； `bit7=0`（无新数据）→ 间隔 **10 ms** 再读一次，不碰 26 字节点位缓冲； `bit7=1` → 才进 `STEP_LOOP` 读坐标并清标记。总线占用比上一版（无条件每

3.3 ms 读 26 字节）小一个数量级。

2. **INT 快通道**： `INT` 拉低时跳过轮询等待立刻发起一次请求（`rdy_to<=0; m_req<=1`）， 保证"按下到出准星"不被 10 ms 轮询周期拖后腿。
3. **降速轮询**：空闲轮询周期 ~10 ms（GT911 典型刷新 5–10 ms）。
4. **★ 多点解码修复（2026-10-15 上板复测后新增，重要）**：GT911 **每个触摸点占 8 字节**（权威依据：GT911 Programming Guide 的坐标寄存器表 / Linux `goodix.c` 的 `GOODIX_CONTACT_SIZE = 8` / Slint `gt911` crate 的 `TOUCHPOINT_ENTRY_LEN = 8`）。 点 0 起始寄存器 `0x814F`，之后每点 `+8`： `+0` track id、`+1/+2` X（小端）、 `+3/+4` Y（小端）、`+5/+6` 面积、`+7` 保留。 旧代码按"**每点 5 字节**"解码，点 1 及以后全部错位 —— 上板实测 `x1` 恒为 `0`、 `y1` 恒为 `1025`（`0x401`，正是"点1 X 低字节 + 点1 track id"拼出来的假数）。 更糟的是**仿真假从机当时也是按 5 字节摆的数据**，等于把错误假设一起"验证"了： 仿真 8/8 全绿、真机多点全错。 现在的读法： `STEP_LOOP` 从 `0x814E` 读 **15 字 = 30 字节**（状态字 + 点 0..3，点 3 的 Y 高字节正好落在 `0x816B`），新增 `STEP_LOOP4` 在 `0x8170` 补读 2 字拿点 4 （`b33` 起才是点 4，超出一轮长读的范围）。仿真模型、测试台同步改成 8 字节布局 + 5 点断言（`tch_num = 5`、5 组坐标全对）。
5. **抬手兜底清点**：GT911 有时不产生"抬手"中断， `0x814E` 会一直保持 `0x00`； 而点数据只在 `STEP_LOOP` 完成时才锁存 ⇒ `tch_num` /准星会永远停在最后一帧（上板实测抬手后 `tch_num` 恒为 `2`、 `T1=(193,369)` 的准星挂在屏上不动）。现在 `STEP_POLL` 里加了 `nodata_cnt`： 连续 48 次（≈48 × 10 ms ≈ 0.5 s）读不到任何报点，就主动把 `tch_num` / `tch_valid` 清零。
6. **★ 卡死自救 + 遥测诊断位（2026-10-15 上板复测后新增，重要）**：上板出现"**手指一碰就永久停摆**" —— `reads_ps` 掉 0、 `tch_num` /坐标冻结在最后一帧、 `err_ps` 也一直是 0。 `err_ps=0` 是关键线索：NACK 与超时都不计，说明 IIC 传输**根本没有结束**（`done` 不来）。 逐行穿读后确认 `i2c_master_seq.v` 的看门狗有**两处独立缺陷**（任一都足以让它永不触发）： - 原判据写作 `stop_cnt >= BUSY_LIMIT[15:0]`，把 25 位阈值 `25'd2_000_000`（40 ms）截断成 低 16 位 = 34048（0.68 ms）； - 更致命的是 `phc < QUARTER-1` 分支里每拍 `stop_cnt <= 16'd0;`，而一个相位长度只有 `CLK_DIV = 62` 拍 ⇒ 计数永远到不了阈值。 于是"某状态卡住/被从机拖住 → `done` 永不产生 → 上层 `ST_WAIT` 死等"这条路一旦发生就 **无法恢复**。现在： - `i2c_master_seq.v`： `stop_cnt` 加宽到 25 位，并改成**按"状态停留时间"计数**——新增 `state_q`， `state != state_q` 时清零并锁存，否则自增（`25'h1FFFFFF` 饱和），判据用全宽。 ★ 触发条件还必须带 `state == state_q`：刚换状态的那一拍 `stop_cnt` 还是上一个状态的旧值， 会立刻误触发（自检台抓到过： `S_STOP` 只活 1 拍就被踢回 `S_RECOVER`，两个状态乒乓， `done` 永远不出来）。 触发后进新增的 `S_RECOVER` 状态——**不看总线电平**，纯打 10 个 SCL 时钟把 被拉死的从机"震"出来，再发 STOP 回 `S_END` （`to_r=1`，上报超时）。 - `touch_i2c_ctrl.v`： `ST_WAIT` 等 `done` 超过 `WAIT_LIMIT = 50 ms` 就强制回 `ST_IDLE` 并落到 `STEP_POLL`，并计一次 `stat_timeout` （以后不会再出现"错误计数也是 0"的哑死）。 - 两者叠加 = 无论 IIC 主机因为什么停住，最多 50 ms 后整套触摸轮询自动恢复。 - 第 9 个自检台 `lcd_test2/sim/tb_i2c_watchdog.v` 把这条路径钉死：卡住的传输必须给出 `done + timeout=1 + busy=0`，**并且第二次请求仍能正常完成**（证明是"恢复"而不是"僵住"）。
7. **遥测诊断位**：原来 byte4/5/6 里的 `mode` / `paused` / `tch_type` 一共只用了 4 个位， 其余空位现在上报两套状态机的 `state` / `step` / `busy` （位定义见 **§5.1**）。 目的是让"卡在哪一步"直接可从串口读出，而不用再靠推理。 `lcd2_uart_check.py` 已同步解析。

F6 第 3 项"配置表写入"：留了编译期开关，功能本身未实现

`lcd_test2/rtl/touch_i2c_ctrl.v` 顶部有一个参数开关（本文件 4 处引用： `ST_IDLE` 请求门控、 `STEP_GT_ID` 的后继、新增的 `STEP_CFG` 分支、状态编号 `STEP_CFG = 4'd9`）：

```
parameter CFG_WRITE_EN = 1'b0;   // 0 = 不写配置表（本版默认，用芯片出厂配置）
```

- **默认 `1'b0`**：识别出 GT911 后直接走 `STEP_GT_INFO → STEP_POLL`，也就是上面这套已验证的流程。 综合时这个常量会把新增的请求门控整条折叠掉，逻辑与"没装开关"时一致。
- **拨到 `1'b1`**：识别成功后先进 `STEP_CFG`。该分支目前**故意不发任何 IIC 请求**、把 `step` 停在原地， 所以"开关打开了但功能没写"会立刻表现为**触摸轮询停住**（遥测 `reads_ps` 掉到 0、点数恒 0）， 而不是把未知数据写到 `0x8047` 把芯片配坏。真要补实现：在 `STEP_CFG` 里写原厂 184 字节配置表 （必要时分批，每批后留出配置加载时间），写完 `step <= STEP_GT_INFO` 接回原链。
- **为什么不留"随便一版"的配置表**：写错会把芯片配到错误的分辨率/中断模式，比不写更糟 —— 本版正是靠出厂默认配置拿到 `tch_type=1 (GT911)`、 `reads≈98/s`、 `err=0` 上板结果的。 上板后若仍不报点，这仍是下一步第一候选，但必须先拿到与该屏模组匹配、且经真机验证的配置表。

---

7. 自检测试台（`lcd_test2/sim/`）

测试台	覆盖
`tb_lcd_timing.v`	行/场时序：每行 1344 拍、635 行、614400 有效像素、同步宽度
`tb_screen_pattern.v`	F1 色序与 58 帧换色、四角小字、64/128 网格、准星几何（臂长 +16 截断、编号、5 点颜色）、抬手保留 31 帧后消失、面板与刻度、暂停冻结
`tb_key_ctrl.v`	消抖（<10 ms 抖动不触发）、短按切模式、长按只发一次 paused、脉冲个数
`tb_touch_i2c.v`	假 GT911 / 假 FT5206 从机：识别、产品 ID、分辨率、按需读、坐标解码；（沿用上一版）
`tb_uart_frame.v`	旧版 16 字节帧（沿用）
`tb_uart_frame26.v`	v3 26 字节帧逐字节 8N1 解码 + 帧内 busy 标志
`tb_lcd2_top.v`	顶层集成：时序、IIC 锁定、坐标、按键翻模式、**画面颜色多样性**（抓"一直是纯色"）、UART 有输出
`tb_i2c_watchdog.v`	IIC 主机看门狗：被从机拖住的传输必须 `done + timeout=1 + busy=0`，且**下一次请求仍能正常完成**；用正常 `CLK_DIV` 参数验证不误触发

运行：`lcd_test2/tools/sim_lcd_test2.tcl`（9 个运行项 = 8 个测试台，其中 `tb_touch_i2c` 跑 GT911 / FT5206 两种假从机；全绿判据 `TEST PASSED: 0 errors`）。

---

8. 运行方法

```
# 仿真（8 个自检）
vivado -mode batch -source lcd_test2\tools\sim_lcd_test2.tcl

# 综合验证（含 IOBUF_CHECK）
vivado -mode batch -source lcd_test2\tools\synth_lcd_test2.tcl

# 实现 + 出位流
vivado -mode batch -source lcd_test2\tools\impl_lcd_test2.tcl

# JTAG 下载（需接好 JTAG，且先插好 40P 排线）
vivado -mode batch -source lcd_test2\tools\program_lcd_test2.tcl

# 上位机遥测
E:\Anaconda\python.exe lcd_test2\tools\lcd2_uart_check.py COM3

# 上位机：引导式上板验收（推荐；按提示按键/点屏，自动判定并出表）
E:\Anaconda\python.exe lcd_test2\tools\lcd2_board_check.py --port COM3

# 上位机：触摸专项四阶段探测（验收卡在"多点"时先跑这个，按提示静止/单指/抬手/多指）
E:\Anaconda\python.exe lcd_test2\tools\lcd2_touch_probe.py --port COM3
E:\Anaconda\python.exe lcd_test2\tools\lcd2_touch_probe.py --port COM3 --quick   # 时长减半
```

`lcd2_board_check.py` 的自动判定项（遥测能证明的）：F1 色轮播在走、屏 ID、F6 轮询速率 （要求 60…140 次/秒且 0 错误）、无人触屏时点数为 0、短按切模式并能切回、单指坐标跟手 （≥3 个不同坐标且都在 0…1023 / 0…599 内）、抬手归零、多点 ≥2、长按暂停并能恢复。 只有"准星画得对不对 / 抬手保留 0.5 s 的观感"它看不见，会在结尾单独列成待目视项。

8.1 本机运行环境注意（都踩过）

1. **Vivado 不在 PATH 上**，每条命令前都要： `$env:Path='D:\Vivado\2026.1\Vivado\bin;'+$env:Path` 。
2. **`tools/run_vivado.ps1` 不能在这里 dot-source**（在受限语言模式下会报 `DotSourceNotSupported` ），所以要自己把环境摆好再起 Vivado： `HOME/USERPROFILE=$root` 、 `APPDATA=LOCALAPPDATA=$root\.appdata` 、 `TEMP=TMP=$root\.tmp` 、 `XILINXD_LICENSE_FILE=D:\Vivado\Xlnx_2026_1.lic` ，然后 `& vivado -mode batch -nojournal -nolog -source <tcl> *> <log>` 。
3. **JTAG 下载必须放开权限跑**：受限沙箱下 `hw_server` （PyInstaller 打包）解包会失败，报 `[PYI-4252:ERROR] Failed to extract entry: VCRUNTIME140.dll` → `ERROR: [Labtools 27-2220] Launch Error` ，最终 `connect_hw_server` 失败。 同一条命令用完整权限重跑即可（ `LCDPROG PROGRAMMED_OK` ）。
4. **`xvlog` 不展开 `*.v` 通配符**，要把文件全名列表拼好再传（ `& xvlog @files` ）。 另外**先单独建好工作目录再跑 xvlog**：在同一条命令里新建目录再调用，xvlog 会报 `CRITICAL WARNING: [Common 17-183] Failed to open handle xvlog.log` 。
5. **Vivado 有时在退出阶段卡死**：日志停在 `Bitgen Completed Successfully.` 、进程 CPU 跑满。 此时位流与报告都已落盘完整，核对 `_lcd_work/impl_lcd2/` 下的 `.bit` / `timing.rpt` / `util.rpt` 后直接结束进程即可，不必重跑。
6. **xsim 的连续赋值不会跟踪"函数里的隐式自由变量"**：把模块级信号（ `pix_x/pix_y` ）在函数里直接读、再用连续赋值调用该函数时，xsim 不会把函数体加进该赋值的敏感表，仿真里准星一直不画（综合/上板

却正常）。 教训：**函数要显式传参**（本版 `cross_hit/lbl_hit/f1_*_ch/tick3` 全部改成显式传 `px,py,...`）。

---

9. 验证记录

9.1 上板验收清单（按顺序做，每步记结果）

#	步骤	期望	结果
1	断电，插好 40P 排线，上电，接 JTAG	屏不热、不闪白	
2	下载 `lcd_test2_top.bit`	`PROGRAMMED_OK`	✅ 2026-10-15 00:52，540074 字节 → `xc7a35t_0`（IDCODE `0362D093`，`localhost:3121/xilinx_tcf/Digilent/210512180081`），日志 `_lcd_work/prog_lcd2.log`
3	看屏：上电后 F1	8 色满屏循环，每色约 1 s，白/黑/红/绿/蓝/青/品红/黄 顺序正确	⬜ 待人工看屏（遥测侧已见 `color_idx` 每约 1 s 递增循环，见 9.5）
4	看四角小字	左下色名+序号、右上 RGB 十六进制、右下 `FRAME` 计数在跳	⬜ 待人工看屏
5	短按 `key_n` 一次	切到 F2：64/128 网格 + 底部/右侧刻度 + 左上信息面板	⬜ 待人工短按
6	面板第 2 行	`PTS 0 IC GT911`（若为 `IC ---` 说明触摸 IC 初始化没过）	✅ 遥测已证 `tch_type=1 (GT911)`、`panel=ATK-7016 1024x600`、`0x814E=0x00`、`reads≈98/s`、`err=0`
7	单指按屏	触点出现白色十字+方框 + `T1`，位置与手指一致（误差 ≤ 几个像素）	
8	五指同时按	出现 5 个不同颜色准星 T1..T5，编号与面板 `PTS n` 一致	
9	抬手	准星保留约 0.5 s 后消失，面板保留最后坐标	
10	长按 1 s	`paused` 置位：色轮播/计数冻结	
11	再长按 1 s	恢复	
12	短按回 F1	网格消失、色轮播从当前色继续	

#	步骤	期望	结果
13	COM3 运行 `lcd2_uart_check.py`	每秒 1 帧、26 字节、帧头 `A5 4C 03`，`err=0`	✅ 8 s 收 8 帧 / 208 字节 / `bad_frames=0` / `err=0` / `reads≈98`，`RESULT: PASS`
14	边点屏边看遥测	`pts` / `T1 x,y` 跟手指走，`reads` 约 100 次/秒量级（10 ms 轮询）	⬜ 待人工点屏（本次窗口没人碰屏：`pts=0`、`tch_valid=0`、`status=0x00`）

> 上表第 4–5、7–12、14 步都可以交给 `lcd_test2/tools/lcd2_board_check.py`：它一步步提示 "短按 / 单指滑动 / 抬手 / 多点按住 / 长按"，同时从遥测里自动判定（帧完整性、屏 ID、F6 轮询速率、 无人触屏点为 0、模式来回切换、坐标跟手且范围内、抬手归零、多点 ≥2、暂停与恢复）， 结果表写到 `_lcd_work/board_check_result.md`，可直接贴回本表。 它判不了的只有"准星在屏上画得对不对、抬手保留 0.5 s 的观感"——这两项必须目视。

9.2 仿真结果（9/9 通过）

```
RUN: tb_lcd_timing          TEST PASSED: 0 errors
RUN: tb_uart_frame          TEST PASSED: 0 errors
RUN: tb_uart_frame26        TEST PASSED: 0 errors
RUN: tb_key_ctrl            TEST PASSED: 0 errors
RUN: tb_screen_pattern      TEST PASSED: 0 errors
RUN: tb_touch_i2c (GT911)   TEST PASSED: 0 errors
RUN: tb_touch_i2c (FT5206)  TEST PASSED: 0 errors
RUN: tb_lcd2_top (集成)      TEST PASSED: 0 errors
RUN: tb_i2c_watchdog        TEST PASSED: 0 errors
SIM_LCD2_DONE             （xsim 1 分 38 秒）
```

综合：`Synthesis finished with 0 errors, 0 critical warnings`，`IOBUF_CHECK: 6`（= `lcd_rgb[7]/[15]/[23]` + `touch_scl/touch_sda` + `touch_int`，与设计预期一致 —— 只有这 3 根 RGB 需要读回屏 ID，其余 21 根被综合成 OBUF）。

> **2026-10-15 多点解码修复后复跑**：仍全绿（`SIM_LCD2_DONE`）。 其中 `tb_touch_i2c (GT911 假从机)` 的假从机已按真片的 **8 字节/点** 布局摆放 5 个点， 断言也升级为 `tch_num = 5` 且 **5 组坐标全部逐一比对**：点0(600,256) 点1(900,400) 点2(100,500) 点3(700,200) 点4(950,550)（点 4 走单读 `STEP_LOOP4`）。 这一版仿真才真正能拦住"点位解码错位"这类错误 —— 见 §6 第 4 条与 §9.6。

9.3 时序修复（第一版实现 WNS = −2.198 ns，已修）

第一次实现的 `report_timing_summary`：**Setup 18 个失败端点，WNS = −2.198 ns，TNS = −36.26 ns**（Hold 通过）。 最差路径 22.08 ns / 38 级逻辑：

```
Source:      u_pat/frame_cnt_reg[21]/C
Destination: u_pat/pixel_data_reg[2]/D
Logic Levels: 38 (CARRY4=12 LUT2=2 LUT3=3 LUT4=2 LUT5=3 LUT6=16)
```

根因：F1 右下 `FRAME nnnnnn` 的文字是用 `frame_cnt % 1000000` + `÷10/÷100/÷100000` 现算的十进制位， 底部/右侧刻度数字也用 `{pix_x[9:7],7'd0}` 现算 `÷10/÷100`。这些**24 位取模/除法是长进位链（12 个 CARRY4）**， 而且直接串在"帧计数/像素坐标 → 字模查表 → 像素颜色"的组合路径上，50 MHz（20 ns）装不下。

改法（`screen_pattern.v`，两处都从像素路径里彻底移除除法）：

1. 帧计数改成 6 位独立 BCD 寄存器 `fd0..fd5`（个位…十万位），在 `frame_st` 时做一次 `bc1..bc5` 进位传递的十进制 +1；被除数是**状态**而不是组合函数，进位链变成寄存器间 6 级。
2. 刻度数字改成查表函数 `tick3(idx, col)`（`idx = pix_x[9:7]`，只有 8 种取值 0/128/…/896）， 一次 3 位→8 位查表代替 `÷10/÷100`。

`digit3()` 与像素路径里的 `dg()` 调用一并删除（`dg()` 只留给信息面板的 `pn_dec`，那是每帧一次、不在像素路径）。

> 教训：**任何除法/取模（尤其 16/24 位）都不要出现在像素组合路径里。**要在屏上显示十进制数字， 就用 BCD 状态寄存器或小位宽查表；本设计里"文字"是最高频的组合逻辑，字模查表本身只有 2~3 级。

9.4 实现与位流结果

修复后重跑综合 + 实现（`synth_lcd_test2.tcl` / `impl_lcd_test2.tcl`）：

项	结果
综合	`Synthesis finished with 0 errors, 0 critical warnings and 71 warnings`， `IOBUF_CHECK: 6`
时序（Setup）	**0 失败端点，WNS = +2.870 ns，TNS = 0.000 ns**
时序（Hold）	0 失败端点，WHS = +0.153 ns，THS = 0.000 ns
脉宽	0 失败端点，WPWS = +9.500 ns
位流	`_lcd_work/impl_lcd2/lcd_test2_top.bit`，**540,074 字节**，2026-10-15 00:30:11，SHA256 `F4C69B23…C3E7A1D`
资源	LUT 2769 / 20800（13.31%），FF 1064 / 41600（2.56%），BRAM 0，DSP 0，IOB 37（14.80%）

对照：修改前 Setup WNS = −2.198 ns / 18 个失败端点 → 修改后 +2.870 ns / 0 个失败端点，**净改善 5.07 ns。** 时序余量 2.87 ns（约 14%），后续如果要加画面功能，注意别再把长组合链塞进像素路径。

> 说明：本次实现全部完成后 Vivado 在退出阶段一直空转（CPU 100%、日志停在 `Bitgen Completed Successfully.`）， 位流与报告都已落盘且完整，故手动结束了该进程；综合/实现/位流三项产物均完整可用。

最终版本重建（加入 F6-3「配置表写入」开关 `CFG_WRITE_EN` 之后）：重跑 `synth_lcd_test2.tcl` + `impl_lcd_test2.tcl`， `Synthesis finished with 0 errors, 0 critical warnings and 72 warnings`； 时序 Setup **0 失败端点 WNS = +2.870 ns / TNS = 0.000 ns**，Hold WHS = +0.153 ns，脉宽 WPWS = +9.500 ns； 位流 `_lcd_work/impl_lcd2/lcd_test2_top.bit`，540,074 字节， SHA256 `36AEABACAE0B3B7D63FACF02B34ED6EA86018A2BBC3D524451A1CAD074324CB6`。 位流字节数与上表旧版完全相同——

新增的 `CFG_WRITE_EN` 默认 0，被综合常数折叠，逻辑与上一版等价； **上板用的是这一版。**同日又出现了 9.4 注里那个"退不出去"的现象，同样按产物完整性处理后手动结束进程。

9.5 上板结果

下载：2026-10-15 00:52， `_lcd_work/impl_lcd2/lcd_test2_top.bit` （540,074 字节，SHA256 `F4C69B23…C3E7A1D`） 经 `lcd_test2/tools/program_lcd_test2.tcl` 下到 `xc7a35t_0` （IDCODE `0362D093`）， hw_server `localhost:3121/xilinx_tcf/Digilent/210512180081`，输出 `LCDPROG PROGRAMMED_OK`，日志 `_lcd_work/prog_lcd2.log`。

遥测（COM3 @115200， `E:\Anaconda\python.exe lcd_test2/tools/lcd2_uart_check.py --port COM3 --seconds 8`，2026-10-15 00:54）： `RESULT: PASS`

观测项	实测	判读
帧完整性	8 s 收 8 帧 = 208 字节， `bad_frames=0`	F5 v3 帧格式、波特率、每秒 1 帧全部正确
屏 ID	`id_valid=1` 、one-hot `0x04` → ATK-7016 1024×600	双向 RGB 位读 ID 的通路正常
色轮播	`color_idx` 5 s 内走过 2→3→4→5→1（每秒约 1 步）	F1 轮播在板上真实运行（每色 58 帧 ≈ 1 s）
帧计数	`frame_cnt` 每帧 +59 （1062→1121→1180→1239→…）	屏在 1024×600 @59.4 Hz 稳定扫描
触摸 IC	`tch_type=1 (GT911)` 、 `status 0x814E=0x00` 、 `reads≈97…98/s` 、 `err=0`	F6 按需读改造后轮询正常，总线无 NACK/超时
触点	`tch_valid=0` 、 `points=0` 、 `T1=(0,0)`	没人碰屏时的正确值（ `tch_valid` 的定义是"本次读到的坐标有效"，不是"IC 就绪"）

90 秒连续录制（ `_lcd_work/uart_v3_live.log` ，00:54:11 起）： 收 90 帧 = 2340 字节， `bad_frames=0` ，丢帧 0 —— 每秒一帧的节奏在 90 秒里一个都没漏。 该窗口内 `modes seen = [0]` 、 `paused seen = NO` 、 `frames with points>0 = 0/90` （当时没人在板前操作）。

5 分钟无人值守稳定性录制（最终位流重新下载之后，300 s 窗口）： 收 **298 帧 / 7748 字节，** `bad_frames=0` （欠 2 帧是收发两端起步/收尾各切掉一帧所致）， `reads/s = 98` 、 `errors/s = 0` （F6 的 10 ms 轮询在 5 分钟里一直稳）， `panel id valid=1` one-hot `0x04` 、 `color_idx` 在窗口内持续推进、 `frame_cnt` 从 23010 继续累加、 `mode=0` 、 `paused=0` 、 `points=0/298` 。**这一窗口里没人按键也没人碰屏**（自动判定结果同样 `RESULT: PASS` ）。

引导式验收工具（本轮新增 `lcd_test2/tools/lcd2_board_check.py` ）： 把 9.1 里"要动手"的步骤变成提示 + 自动判定，冒烟测试已跑通（基线 5 s → 5 帧 0 错、 F1 `color_idx` 依次 3→4→5→6→7、屏 ID 正确、 `reads/s=98` 、 `err=0` 、无人触屏 `points=0` ， 并把表格写到 `_lcd_work/board_check_result.md` ）； 人不在板前时那些步骤会如实记 `FAIL` / `SKIP` ， 不会假装通过。

重新下载（最终版）： 2026-10-15 07:06，用同一脚本下载上面 SHA256 `36AEABAC…4CB6` 的位流， `INFO: [Labtools 27-3164] End of startup status: HIGH` → `LCDPROG PROGRAMMED_OK` ， 日志 `_lcd_work/program_lcd2_final.log` 。

> **下载必须放开权限跑**：受限沙箱下 `hw_server`（PyInstaller 打包）解包失败，报 `[PYI-4252:ERROR] Failed to extract entry: VCRUNTIME140.dll` → `ERROR: [Labtools 27-2220] Launch Error`，`connect_hw_server` 起不来；同一条命令用完整权限重跑即成功（详见 8.1）。

重新下载后复测（COM3 @115200，30 秒窗口，2026-10-15 07:07）：`RESULT: PASS` —— `bytes=780 frames=30 bad_frames=0`；`reads per sec = 98`、`errors per sec = 0`（F6 的 ~10 ms 降速轮询 = 约 100 次/秒状态字读取，实测 98、零错，**这是"按需读 + 10 ms 轮询"在板上的直接证据**）；`panel id valid=1` one-hot `0x04`、`mode=0`、`paused=0`、`color_idx=6`、`frame_cnt=3186`、`status 0x814E=0x00`、`points=0/30`（无人触屏）。

仍需人工确认的项（要看着屏动手，对应 9.1 的第 3–5、7–12 步）： F1 八色与四角小字的实际观感；短按切到 F2 后网格/刻度/信息面板是否出现； 点屏后准星与 `T1..T5` 编号是否跟手（这是本版 F3 的重点）； 长按冻结是否生效。

9.6 多点触摸问题：诊断与修复（2026-10-15，用户报"第 5 步卡住"之后）

现象：引导式验收 `lcd2_board_check.py` 的第 5 步（要看到 `tch_num >= 2` 的帧）一直等不到； 而且第 3 步"单指"也没等到点（`tch_num` 保持 0），所以第 4 步被按"未做"跳过。

专项探测（`lcd_test2/tools/lcd2_touch_probe.py`，COM3，150 s，149 帧）：

```
tch_num = 2   （149 帧里恒定不变）        status 0x814E = 0x00（149 帧恒为 0）
T1 = (193,369)                            T2 = (0,1025)   （全程不变）
VERDICT: no touch data at all in this window
```

两条结论：

1. **`T2=(0,1025)` 是解码错位的直接证据**：`1025 = 0x401`，正是"按每点 5 字节"错位后 `{点1 X 低字节, 点1 track id=1}` 拼出来的假数（真值应为 `(900,400)`）。 权威依据与修法见 §6 第 4 条。
2. **`tch_num` 抬手后不清零**：`status` 恒 0 而 `tch_num` 恒定，说明点数据只在 `STEP_LOOP` 里锁存、没有任何路径把它清零。那个 `tch_num=2` 本身来自上电后芯片缓冲里的一次真实报点 （GT 探测阶段读到 `probe_gt = 0x82` = ready + 2 点）——所以**芯片是能报多点的**， 问题在解码与清点。修法见 §6 第 5 条（`nodata_cnt`）。

⚠ **仿真为什么没发现**：假从机（`lcd_test2/sim/iic_slave_model.v`）当时也是按"每点 5 字节" 摆数据的，等于把 RTL 的错误假设**一起验证**了 —— 仿真 8/8 全绿、真机多点全错。 本次已把假从机改成真片的 8 字节/点布局并升到 5 点断言。

复验（本次修改后）： - 仿真：9 个运行项全部 `TEST PASSED: 0 errors`（含新增的 `tb_i2c_watchdog`）；`tb_touch_i2c (GT911)` 逐点比对通过 （点0(600,256) 点1(900,400) 点2(100,500) 点3(700,200) 点4(950,550)，点 4 走 `STEP_LOOP4`）。 - 综合/实现/位流：见下表。 - 上板：用四阶段探测 `lcd2_touch_probe.py`（静止基线 → 单指 → 抬手 → 多指）复测， 结果见本文件 9.5 补记。

项	值
位流	`_lcd_work/impl_lcd2/lcd_test2_top.bit`，524,666 字节，2026-10-15 12:23:57
SHA256	`793408948106A8B309C6E5FCA0F64978E0148CA87F762543A1FEC39199476E53`

项	值
时序	`All user specified timing constraints are met.`（WNS **+3.093 ns**、TNS 0；WHS +0.108；WPWS +9.500；失败端点 0）
资源	LUT 2,773（13.33%） / FF 1,089（2.62%） / BRAM 0 / DSP 0 / IOB 37（14.80%）
综合	0 errors、0 critical warnings
上板	`PROGRAMMED_OK`（`_lcd_work/program_lcd2_wd.log`）；烧后 10 s 遥测 `reads/s = 98`、`err/s = 0`、`RESULT: PASS`

★ 上板实测结论（四阶段探测 `_lcd_work/touch_probe_fix.log`）：多点恢复正常

```
chip-reported points  max = 5      ← GT911 自己写进 0x814E 的点数
tch_num (RTL)         max = 5      ← FPGA 解码后送显示的点数
VERDICT: multi-touch WORKS (chip and RTL both >= 2 points).
```

帧	`0x814E`	芯片点数	`tch_num`	T1	T2
4366	`0x81`	1	1	(646,418)	—
4602	`0x82`	2	2	(466,325)	(708,303)
4779	—	—	**5**	(922,129)	(265,220)

⇒ ① **8 字节/点的解码修对了**；② **GT911 本来就配置成 5 点上报，F6 第 3 项（配置表写入）不需要做**； ③ 用户目视确认"显示和触控现象完全正确"（准星跟手、抬手保留、多点各自编号/颜色、长按冻结/恢复）。

9.7 触摸"一碰就永久停摆"：诊断与修复（2026-10-15 第二次上板复测后）

现象（多点修复版烧进板子后，用户点屏复测）：

```
frame_cnt / color_idx / mode 都在正常推进        ← 画面与遥测链路好着
tch_num = 1、point0 = (411,219) 恒定不变         ← 触摸字段全部冻结
status 0x814E = 0x00 恒定
reads_ps = 0      err_ps = 0                     ← ★ 关键：既不成功、也不失败
```

连续两个 5–6 s 窗口、扳手复位重烧后仍能重现：**手指一碰就再也不恢复**， 且 `point0` 与那次按下的位置对得上 ⇒ 停摆发生在"读到第一个触点之后"。

推理链：

1. `reads_ps` 只统计"成功完成的 IIC 传输"（`stat_reads`）， `err_ps` 只统计 NACK + 超时。 两个都是 0 ⇒ 传输**没有结束**，不是失败： `i2c_master_seq` 的 `done` 不产生， `touch_i2c_ctrl` 的 `ST_WAIT` 在等 `done`，永远等下去。
2. 逐行穿读 `i2c_master_seq.v`，确认看门狗写错了、是死代码（两条独立缺陷）： `stop_cnt >= BUSY_LIMIT[15:0]` 把 25 位阈值截断成 16 位；同一个 `phc < QUARTER-1` 分支里又有 `stop_cnt <= 16'd0;` 把计数每拍清零（相位只有 `CLK_DIV=62` 拍）。 两者任一都足以让看门狗永不触发。
3. 所以"某个状态卡住 / 被从机拖住"（GT911 在长读期间拉长 SCL 是合规行为：它是主机，可以 stretch）会变成**不可恢复的锁死**。仿真复现不了，因为假从机从不 stretch。

修法（都已落盘，9 个自检台全绿）：

文件	改动
`lcd_test2/rtl/i2c_master_seq.v`	`stop_cnt` 25 位 + 改成**按状态停留时间**计数（`state_q` 比对，换状态即清零），判据全宽 `stop_cnt >= BUSY_LIMIT` 且**必须 `state == state_q`**；触发 → 新增 `S_RECOVER` 状态：不看总线电平、纯打 10 个 SCL 时钟 + STOP → `S_END`（`timeout=1` 上报）
`lcd_test2/sim/tb_i2c_watchdog.v`	第 9 个自检台：卡死传输必须 `done + timeout=1`，且第二次请求仍能完成；另用正常 `CLK_DIV` 参数验证不会误触发
`lcd_test2/rtl/touch_i2c_ctrl.v`	`ST_WAIT` 增加 `WAIT_LIMIT = 50 ms` 兜底：超时强制回 `ST_IDLE` + `STEP_POLL` 并计 `stat_timeout`
`lcd_test2/rtl/touch_i2c_ctrl.v` / `lcd_test2_top.v`	新增 16 位 `dbg` 总线，把 touch FSM 的 `state/step` 与 IIC 主机的 `state/busy` 塞进遥测 byte4/5/6 的空位（位定义见 **§5.1**）
`lcd_test2/tools/lcd2_uart_check.py`	逐帧打印 `tstate/step/m_state/busy`，window 汇总增加状态分布直方图

修复过程中的三个自伤 bug（都已被自检台抓住）：① 看门狗分支里没清 `stop_cnt` ⇒ 每拍重复触发、 状态钉在 `S_RECOVER`（换了个卡法）；② 忘了"换状态那一拍 `stop_cnt` 是旧值"⇒ `S_STOP` 只活 1 拍又被 踢回 `S_RECOVER`，两个状态乒乓；③ 自检台里 `done` 之后才采 `nack/timeout`，而 `S_IDLE` 已把它们清零 ⇒ 改成与 `done` 同拍锁存。教训：**超时/看门狗类逻辑必须自检台钉死，"看起来能恢复"和"真的恢复"差距很大。**

这次修复的定位：不是"猜一个补丁"，而是把"不可恢复的等待"全部变成"超时+自动复位"， 同时让遥测把状态机的状态直接暴露出来 —— 下一次上板如果还停，串口会直接告诉我们是 `tstate=WAIT + m_state=?` 还是别的地方。

待上板确认（本文件 9.5 补记）：碰屏后 `reads_ps` 应保持在 ~98/s、 `err_ps=0`， 且**抬手后 0.5 s 内**点数归零；偶尔出现 `m_state=RECOVER` 说明看门狗真的救过一次总线。

---

10. 与需求书的差异 / 未做项

1. **F5 帧长 26 字节**（需求书 V1 写的是 16 字节）：v3 增加了 `paused`、 `color_idx`、24 位 `frame_cnt`、 第 2 点坐标、16 位 `reads_ps` / `err_ps` 诊断计数，便于无人值守时定位问题。
2. **F6 第 3 项（配置表写入）功能未实现，但已按需求留了编译期开关**：`lcd_test2/rtl/touch_i2c_ctrl.v` 顶部 `parameter CFG_WRITE_EN = 1'b0;`（默认关闭，见第 6 节）。默认路径与"没装开关"等价； 拨到 `1'b1` 会显式停在 `STEP_CFG`（现象是触摸轮询停住），避免盲写 `0x8047` 把芯片配坏。 **★ 该项经上板实测确认已无必要**：`lcd2_touch_probe.py` 四阶段探测显示 GT911 自己就往 `0x814E` 里写了 `0x85`（5 点）， FPGA 也解出 `tch_num = 5` —— 芯片本来就配置成 5 点上报， 当初"只报 1 点"的判断来自 RTL 每点 5 字节的错误解码（见 §6 第 4 条与 §9.6）。
3. 屏上文字只有 5×7 英数点阵（放大的也只有 2 倍），没有大字号标题与中文（需求书已明确不做）。
4. 灰阶/单通道 ramp、扩散圆环、拖尾等本轮未要求，均未实现。